PROJECT HANDOFF / WINDOWS DEMO

Portable setup and presentation guide

This guide is for handing the project to someone who will run it on a different Windows computer. The included Docker Compose demo is the recommended setup: it starts the database, message broker, cache, local blockchain, AI API, backend, worker and both web interfaces in containers.

QUICK START

```
node .\scripts\run-demo.js
```

Builds the local services and seeds the synthetic demo.

Main dashboards: localhost:3000 | Field Operator: localhost:5173

> Project scope: This is a software-only demo using synthetic records. It is not a clinical system, medical recommendation, production deployment, real sensor installation, public blockchain, or real-email service.

A practical handoff for a clean-machine setup and a role-by-role project presentation.

Contents

1. Prepare the project ZIP . . . 3
2. Install software on the receiving computer . . . 3
3. Extract and start the demo . . . 4
4. Features by user role . . . 5
5. Suggested 10-15 minute presentation . . . 6
6. Useful commands during the demo . . . 7
7. Common setup issues . . . 7

1. Prepare the project ZIP

Before making the ZIP, keep the source code and configuration templates, and exclude machine-specific files:

- Include: `README.md`, `.env.example`, `docker-compose.yml`, `docker-compose.mqtt.yml`, `docker-compose.test.yml`, `backend/`, `blockchain/`, `frontend/`, `field-operator-panel/`, `ai-services/`, `mosquitto/`, `docker/`, `scripts/`, `docs/`, `ARCHITECTURE.md`, `FUTURE_ARCHITECTURE.md`, and `results/` if you want to include the saved demo reports.
- Exclude: `.env` and any other real `.env` files (including nested ones), `.git/`, `node_modules/`, `venv/`, `.venv/`, `__pycache__/`, `.pytest_cache/`, `build/`, `dist/`, and `coverage/`.
- Keep `.env.example`; it is the template, not the private runtime credentials file. Do not put `.env` in the ZIP or send it in screenshots. The other computer must create its own local secrets.

On Windows, use File Explorer to create the ZIP from a clean copy of the project folder. Before sending it, open the archive and confirm `.env.example` is present and `.env` is absent. `node_modules` and Python virtual environments are not required because Docker builds the app dependencies from the manifests.

2. Install software on the receiving computer

1. Docker Desktop: Install Docker Desktop for Windows using the official Windows installation guide. The default WSL 2 backend is suitable for this Linux-container project. If Windows prompts to enable WSL or virtualization, follow the installer prompts and restart Windows if requested. Launch Docker Desktop and wait until its Engine is running.
2. Node.js 20 or newer: Install the Windows LTS release from the official Node.js download page. Node is used to run the repository's demo launcher; npm is included with the installer.
3. Browser: Use a current Chrome or Edge browser for the role dashboards and QR presentation.
4. Internet: The first launch needs outbound access to download container images and build dependencies. The disposable Ethereal email-capture readiness check also needs internet access.

You do not need to install MongoDB, Redis, Mosquitto, Python, Hardhat, or a Solidity compiler on Windows for the Compose demo. Compose runs those services in containers.

Open PowerShell and confirm the installations:

```
node --version
npm --version
docker --version
docker compose version
```

If the Docker commands fail, open Docker Desktop and wait for its Engine before continuing.

3. Extract and start the demo

1. Extract the ZIP to a writable folder, for example `C:\Projects\vaccine-supply-chain-system`. Avoid extracting into the ZIP preview window.
2. Open PowerShell in the extracted project root (the folder containing `package.json` and `docker-compose.yml`):

```
Set-Location 'C:\Projects\vaccine-supply-chain-system'
```

3. Start the complete demo and create the synthetic demo accounts/data:

```
node .\scripts\run-demo.js
```

On the first run this builds the images, starts the services, waits for Compose health checks, then seeds the demo. It creates a fresh ignored root `.env` with local-only secrets if `.env` does not already exist; it does not overwrite an existing `.env`. Keep the PowerShell window open while it runs. The first build may take several minutes.

4. Confirm the services are running:

```
docker compose ps
```

5. Open the dashboards:

- Main dashboards (Admin, Manufacturer, Distributor, Healthcare): http://localhost:3000
- Separate Field Operator panel: http://localhost:5173
- API health page: http://localhost:5000

Demo logins

The seeded accounts use the following email and role combinations. Choose the matching role on the login page. All five use the same `DEMO_SEED_PASSWORD` value in that machine's root `.env`.

Role	Login email	Open this UI
Admin	`admin@demo.synthetic.test`	Main dashboard, port 3000
Manufacturer	`manufacturer@demo.synthetic.test`	Main dashboard, port 3000
Distributor	`distributor@demo.synthetic.test`	Main dashboard, port 3000
Healthcare Professional	`healthcare@demo.synthetic.test`	Main dashboard, port 3000
Field Operator	`fieldoperator@demo.synthetic.test`	Field Operator panel, port 5173

The password is generated for that computer and is not printed by the launcher. To retrieve it locally, run this command in the project root before screen-sharing:

```
(Get-Content .\.env | Where-Object { $_ -match '^DEMO_SEED_PASSWORD=' }) -replace
'^DEMO_SEED_PASSWORD=', ''
```

Share that password only with the people who need the demo login. Do not put it in the ZIP, slides, screenshots, or a public repository. Preserve `.env` for that local demo because the seed does not reset existing users' passwords. On a fresh setup with an empty database, the launcher's newly generated password will work for the newly seeded accounts.

For a multi-role presentation, keep each role in a separate browser tab or profile and sign in with that role's matching email and the same demo password. The Field Operator signs in through the separate panel.

4. Features by user role

The backend scopes records and actions by account, organization, shipment assignment, and permission. Different roles can therefore see different batches, shipments, and alerts.

Admin

- Checks local service readiness, including API, MongoDB, MQTT, Redis, AI, local Hardhat chain/contract, and test-email capture.
- Reviews batches, policies, inventory, orders, alerts, audit history, traceability, forecasts, reports, and synthetic feedback analytics.
- Reviews/approves or declines eligible healthcare orders and assigns eligible stock and a distributor.
- Previews supply-planning recommendations and manages approved demo policies and authorized alert/quality-review actions.
- Demonstrates the simulator, manual sensor-reading entry, and end-to-end event history.
- The user directory is read-only; account editing is not implemented.

Manufacturer

- Views its account-scoped batches, manufacturing history, production reports, inventory certificates, shipment context, alerts, and analytics.
- Creates a synthetic batch and proceeds through separate Production and Lab Testing sign-offs; a created batch is not automatically released.
- Reviews batch policies and designated reviewer actions where authorized.
- Runs virtual sensor scenarios and views telemetry history, traceability, local-chain audit details, and batch QR payloads.
- Inventory certificates are demo records, not clinical quality certificates.

Distributor

- Views assigned shipments, shipment states, route context, batch information, scoped alerts, and shipment history/reports.
- Advances an eligible shipment through Verify batch -> Dispatch shipment -> Supply hospital.
- Can enter a synthetic manual sensor reading and inspect relevant shipment telemetry.
- The backend blocks dispatch or supply when ownership, batch release, product, quantity, or cold-chain checks fail.

Healthcare Professional

- Requests synthetic vaccine stock through the order workflow and reviews incoming deliveries/history.
- Confirms receipt only after an eligible shipment is supplied; confirmed receipts are the source for hospital stock.
- Verifies batch identity and, when available, separately checks sensor evidence anchored to the local demo chain.
- Uses reports, feedback, and optional synthetic administration workflows. Use invented references only; never enter patient data.
- Can import CSV telemetry where available, but cannot submit a manual sensor reading through the manual-reading form.

Field Operator

- Uses the separate field panel to inspect assigned shipment context, telemetry history, alert information, and location/sensor readings.
- Can submit a synthetic manual reading with a required explanatory note where the assignment permits it.
- Does not create batches, approve manufacturing release, allocate stock, or manage system-wide policies.

5. Suggested 10-15 minute presentation

The full click-by-click script is in Viva demonstration guide. This short sequence gives each role a clear purpose:

1. Introduce the scope (1 minute). Explain that sensor, GPS, order, inventory, and demand data are synthetic. Approved batch policy rules determine safety alerts and holds; AI output is advisory.
2. Admin (2 minutes). Show readiness, a seeded batch and its displayed demo policy, inventory, synthetic forecast provenance, and order-review/supply-planning areas. Explain that a fresh seed's batches are unreleased, so the planning preview may correctly allocate zero stock.
3. Manufacturer (2 minutes). Show a batch and its history. Optionally create a uniquely named synthetic batch and demonstrate separate Production and Lab Testing sign-offs. Show its QR only if a verification payload is available.
4. Admin or Manufacturer (2 minutes). Use an existing batch with an approved demo policy. In the virtual sensor panel, select or create a device and generate a `NORMAL` reading. Show the saved chart and simulated route.
5. Admin or Distributor (2 minutes). Submit one manual synthetic reading above the displayed policy range (for the seeded demo range, `11` °C is an example) and include an explanatory note. Show the policy alert, source label, reason, and timestamp. Do not double-submit while the first reading is processing.
6. Distributor and Healthcare (2-3 minutes). Show the assigned-shipment view and the guarded verify/dispatch/supply sequence. Switch to Healthcare to show the order, delivery, and QR-verification views. Do not claim a successful delivery if no eligible released batch exists.
7. Field Operator / wrap-up (1 minute). Open the separate panel and show the assigned-shipment sensor view. Close by distinguishing batch identity from anchored sensor evidence and restating that this is a local synthetic demo.

If role sessions are unavailable or a workflow is blocked by release/assignment rules, show the relevant role's screen and explain the guard. Do not edit MongoDB directly or claim a blocked action succeeded.

6. Useful commands during the demo

Run from the project root:

```
# Check service state
docker compose ps

# Inspect recent service logs
docker compose logs --tail 80 backend worker edge-gateway ai-service

# Re-run the repeatable demo seed after the stack is up
docker compose exec -T backend node scripts/seed.js

# Stop the services when finished; persistent database volumes remain
docker compose down
```

Avoid restarting the `hardhat` service while presenting: its chain is in memory, so restarting it discards the chain history even if MongoDB still contains demo records. Do not run `docker compose down -v` unless you deliberately intend to delete persistent demo data and volumes.

7. Common setup issues

Symptom	What to check
`docker` is not recognized or cannot connect	Install Docker Desktop, start it, and wait for the Engine. Close and reopen PowerShell after installing Docker.
Image pulls/build fail	Confirm internet access, disk space, and Docker Desktop status; rerun `node .\scripts\run-demo.js`.
A dashboard URL does not load	Run `docker compose ps`; wait for services to become healthy. Inspect logs with the command above.
Port is already in use	Close the other local service using the port, then restart the demo. Default host ports are 3000, 5173, 5000, 27017, 1883, 6379, 8545, and 8000.
Login rejected	Check the email and selected role, use the machine-local `DEMO_SEED_PASSWORD`, and confirm the seed step completed.
A role cannot see a batch or alert	Check account ownership, shipment assignment, recipient mapping, and organization scope; visibility can differ by role.
QR evidence is pending/unavailable	Wait for the demo Merkle window and refresh. A batch-only verification does not prove anchored sensor history.
Email readiness is degraded	The service uses a temporary Ethereal capture mailbox and needs outbound internet; it does not send real email.

For service and feature details, see Current implementation state, Reliable local platform setup, Operational dashboard guide, and Traceability and demo features.